Literature Review — Distributed Systems Lab

Optimized Task Offloading in IoT–Fog–Cloud Systems: A Comprehensive Literature Review of Metaheuristic Optimization, Deep Reinforcement Learning, SDN, and Mobility-Aware Computation Offloading

---

Keywords: Task offloading, IoT, Fog computing, NSGA-II, Deep reinforcement learning, SDN, DAG, Vehicular networks, Mobility-aware offloading, Proactive handoff

Team: imed eddine benmadi ,

April 2026

Prepared for: Distributed Systems Lab — Literature Review Assignment

Abstract

The rapid proliferation of Internet of Things (IoT) devices in smart-city and vehicular environments has created an urgent demand for low-latency, energy-efficient computation at the network edge. Task offloading—the process of delegating computation from resource-constrained IoT devices to fog or cloud servers—has emerged as the central mechanism for satisfying these demands within the three-tier IoT–Fog–Cloud hierarchy. This paper presents a structured literature review of ten representative works spanning 2021–2025, covering five distinct research directions: (i) multi-objective metaheuristic optimization via NSGA-II with Minimax Differential Evolution, (ii) task pre-filtering through the TOF-Broker Execution Cost framework, (iii) software-defined networking for programmable traffic management, (iv) microservice-oriented DAG task modeling, and (v) mobility-aware offloading with deep reinforcement learning and proactive handoff prediction. We identify the state of the art in each sub-domain, synthesize the research trajectory, and formally define five open research gaps that no published system has jointly resolved. Our analysis argues that the convergence of offline Pareto optimization and online reinforcement learning—applied to DAG workloads with GPS-driven mobility prediction—represents the most promising direction for next-generation vehicular edge computing systems.

Contents

1 Introduction **3**

2 Background and Formal Definitions **4**
2.1 Three-Tier IoT–Fog–Cloud Architecture 4
2.2 Task Offloading Formulation 4
2.3 Execution Cost and Boulder/Pebble Classification 4
2.4 Pareto Dominance and Non-Dominated Sorting 5
2.5 DAG Task Model 5

3 Research Evolution: Four Generations **5**
3.1 Generation 1: Single-Objective Heuristics (pre-2021) 5
3.2 Generation 2: Multi-Objective Evolutionary Algorithms (2021–2023) . . . 5
3.3 Generation 3: Pre-Filtering and Hybrid Systems (2024) 6
3.4 Generation 4: Adaptive AI-Driven Systems (2024–2025) 6

4 State of the Art **6**
4.1 Multi-Objective Metaheuristic Optimization 6
4.2 Task Pre-Filtering: The TOF-Broker 7
4.3 Software-Defined Networking for Fog Systems 7
4.4 Microservice Architecture and DAG Task Modeling 7
4.5 Deep Reinforcement Learning for Task Offloading 7
4.5.1 DRL for Vehicular Fog Computing 7
4.5.2 SDN-Integrated DRL: CSFD 8
4.5.3 DAG Task Offloading with Meta-RL and NSGA-II Pre-training . . 8

4.6 Mobility-Aware Offloading 8
4.6.1 Reactive Handoff: NTB/HTB Dual-Buffer System 8
4.6.2 Proactive Handoff: MAVTO 8
4.6.3 Spatiotemporal Prediction: Digital Twin + STGNN 9
4.7 Comparative Summary 9

5 Open Research Gaps 10

6 Proposed Architecture: Predictive Cloud-Native Mobile Edge 11
6.1 Architecture Overview 11
6.2 Hybrid Offline–Online AI Pipeline 12

7 Comparative Discussion 12
7.1 Against Generation-2 Systems 12
7.2 Against DAG+MRL [8] 12
7.3 Against CSFD [7] 12
7.4 Against MAVTO [9] 13

8 Conclusion 13

1 Introduction

The Internet of Things ecosystem is undergoing unprecedented growth, with tens of billions of connected devices projected by 2030 [1]. A growing proportion of these devices operate in latency-sensitive, mobility-intensive environments: smart vehicles performing real-time collision avoidance, autonomous drones executing path-planning algorithms, and industrial robots running precision control loops. These workloads share a critical characteristic—they generate computation-intensive tasks that exceed the processing capacity of the originating device yet must produce results within strict timing constraints measured in tens of milliseconds.

Conventional cloud computing provides near-unlimited computational resources but introduces WAN propagation delays of 50–200 ms, rendering it unsuitable for safety-critical vehicular applications [1]. Fog computing addresses this by positioning intermediate compute nodes one or two network hops from IoT devices, reducing latency to 1–10 ms. The resulting *three-tier hierarchy* (IoT → Fog → Cloud) creates a fundamental optimization problem: for each incoming task, which tier executes it, under what scheduling constraints, and at what energy cost?

Task offloading—the decision process routing computation across tiers—is formally NP-hard [11]. At IoT scale, the search space is intractable for exact algorithms. Five interconnected challenges define the current research landscape:

C1. Boulder Problem. Compute-heavy tasks routed naively to fog nodes consume disproportionate resources and cause queue cascade failures [2].

C2. Cold-Start Problem. Deep reinforcement learning agents produce near-random policies during early training, causing service degradation before they converge [6].

C3. Static Optimization Problem. Evolutionary optimizers like NSGA-II converge over hundreds of generations—orders of magnitude too slow for millisecond real-time decisions [8].

C4. Network Congestion Problem. Reactive SDN routing cannot respond fast enough to burst traffic events, causing packet loss and queuing delays [3, 7].

C5. Mobility Problem. In vehicular networks, devices traverse fog coverage zones faster than tasks complete, causing mid-computation handoff failures [5, 9].

No single published system resolves all five challenges in a unified framework. This literature review maps the research evolution from single-objective heuristics to adaptive AI-driven systems, identifies the specific gaps that remain open, and positions a proposed architecture—the Predictive Cloud-Native Mobile Edge (PCNME)—as the natural convergence point of this trajectory.

Paper organization. Section 2 establishes technical background and formal definitions. Section 3 traces four generations of research evolution. Section 4 surveys the state of the art in each sub-domain with our ten primary references. Section 5 formalizes five open research gaps. Section 6 describes the proposed unified architecture. Section 7 provides comparative analysis. Section 8 concludes.

2 Background and Formal Definitions

2.1 Three-Tier IoT–Fog–Cloud Architecture

The three-tier model partitions computation infrastructure into: **Layer 1** (IoT devices)—resource-constrained task generators with limited CPU, memory, and battery; **Layer 2** (fog nodes)—intermediate edge servers co-located with network access points, offering low-latency compute within a coverage radius of 150–300 m in urban 5G deployments; and **Layer 3** (cloud)—centralized high-throughput servers providing batch computation at the cost of 20–50 ms additional WAN propagation delay.

2.2 Task Offloading Formulation

Let $\mathcal{T} = \{t_1, t_2, \ldots, t_N\}$ be a batch of N tasks arriving from IoT devices. Each task t_i is characterized by:

$$t_i = \langle \ell_i,\ d_i,\ D_i,\ \mathcal{E}_i \rangle \tag{1}$$

where ℓ_i is the computational demand in Million Instructions (MI), d_i is the input data size in kilobytes, D_i is the end-to-end deadline in milliseconds, and $\mathcal{E}_i$ is the device transmission energy in joules.

The offloading decision assigns each task to a destination $x_i \in \{\mathrm{FOG}_1, \ldots, \mathrm{FOG}_K, \mathrm{CLOUD}\}$. The two primary optimization objectives are:

$$f_1(\mathbf{x}) = \sum_{i=1}^{N} E_i(x_i) \quad \text{(total energy, minimize)} \tag{2}$$

$$f_2(\mathbf{x}) = \sum_{i=1}^{N} L_i(x_i) \quad \text{(total latency, minimize)} \tag{3}$$

subject to:

$$L_i(x_i) \le D_i \quad \forall\, i \quad \text{(deadline constraint)} \tag{4}$$

$$\sum_{i:\, x_i = k} \ell_i \le C_k \quad \forall\, k \quad \text{(fog capacity constraint)} \tag{5}$$

where C_k is the MIPS capacity of fog node k.

2.3 Execution Cost and Boulder/Pebble Classification

The *Execution Cost* (EC) metric [2] determines fog feasibility:

Definition (Execution Cost):

$$\mathrm{EC}(t_i) = \frac{\ell_i}{\mathrm{fog_MIPS}} \quad \text{(seconds)} \tag{6}$$

Task t_i is a *Pebble* if $\mathrm{EC}(t_i) < \theta$, and a *Boulder* if $\mathrm{EC}(t_i) \ge \theta$, where $\theta = 1.0\,\mathrm{s}$ is the default threshold [2].

2.4 Pareto Dominance and Non-Dominated Sorting

Definition (Pareto dominance): Solution $\mathbf{x}^{(a)}$ dominates $\mathbf{x}^{(b)}$, written $\mathbf{x}^{(a)} \prec \mathbf{x}^{(b)}$, iff:

$$f_k\big(\mathbf{x}^{(a)}\big) \leq f_k\big(\mathbf{x}^{(b)}\big) \;\; \forall k, \quad \exists j : \; f_j\big(\mathbf{x}^{(a)}\big) < f_j\big(\mathbf{x}^{(b)}\big) \tag{7}$$

The Pareto front $\mathcal{P}^*$ is the set of all non-dominated solutions. NSGA-II [1] approximates $\mathcal{P}^*$ by evolving a population of candidate routing plans across multiple generations via selection, crossover, and mutation operators.

2.5 DAG Task Model

Modern IoT applications decompose into chains of dependent processing steps [4]. The Directed Acyclic Graph (DAG) model represents an application as $G = (\mathcal{V}, \mathcal{E})$ where node $v_i \in \mathcal{V}$ is a microservice step with properties $\langle \ell_i, d_i^{\text{in}}, d_i^{\text{out}} \rangle$, and edge $(v_i, v_j) \in \mathcal{E}$ encodes data dependency: step v_j cannot start until v_i completes and transmits its output. Total application latency over the critical path is:

$$L_{\text{app}} = \sum_{v_i \in \text{critical path}} \Big[T_{\text{exec}}(v_i) + T_{\text{transfer}}(v_i, v_{i+1}) \Big] \tag{8}$$

where $T_{\text{transfer}} = 0$ when consecutive steps are co-located.

3 Research Evolution: Four Generations

3.1 Generation 1: Single-Objective Heuristics (pre-2021)

Early task offloading research modeled the decision as binary: execute locally or offload entirely to the cloud. Solutions used greedy algorithms, game theory, and single-objective metaheuristics such as Particle Swarm Optimization and Ant Colony Optimization [1]. These approaches optimized one metric while ignoring the other, and operated on static topologies with no mobility modeling.

Key limitation: Single-objective formulations systematically underperform because energy and latency conflict. Minimizing latency alone drives all tasks to cloud (high energy); minimizing energy alone keeps tasks local (high latency). A balanced solution requires explicit multi-objective treatment.

3.2 Generation 2: Multi-Objective Evolutionary Algorithms (2021–2023)

Jafari and Rezvani [1] established the multi-objective framing by applying NSGA-II with Minimax Differential Evolution (MMDE) mutation to the IoT–Fog–Cloud offloading problem. Their binary chromosome encodes routing decisions for a task batch; solutions are evaluated on energy (2) and latency (3) objectives simultaneously, producing Pareto fronts that expose the full energy–latency trade-off space.

The MMDE operator [1] replaces standard random mutation with directional differential evolution:

$$\mathbf{V} = \mathbf{r}_1 + F \cdot (\mathbf{r}_2 - \mathbf{r}_3), \quad F = 0.5, \; \text{CR} = 0.9 \tag{9}$$

where $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3$ are three distinct population members. This improves convergence speed and solution robustness against variable network conditions.

Key limitation: NSGA-II evaluates all tasks—including compute-infeasible boulders—on fog nodes, wasting CPU cycles and causing queue crashes. Additionally, hundreds-of-generations batch convergence is incompatible with millisecond real-time requirements.

3.3 Generation 3: Pre-Filtering and Hybrid Systems (2024)

Nouri et al. [2] introduced the TOF-Broker atop NSGA-II. By computing EC (6) for each task before optimization, the broker eliminates boulders from the search space, reducing problem size by 15–30% and achieving 12.18% energy improvement over plain NSGA-II with negligible latency overhead. Concurrently, the microservice research community [4] established that DAG-structured task models enable partial offloading—running lightweight steps on fog while heavy steps execute on cloud—impossible under the monolithic task model.

Key limitation: The resulting systems remain static. Pre-filtered NSGA-II produces optimized batch routing plans that cannot adapt to changing fog loads, varying network conditions, or moving devices.

3.4 Generation 4: Adaptive AI-Driven Systems (2024–2025)

The current generation replaces or augments offline optimization with online learning agents. For vehicular settings, DRL-based offloading [6] demonstrated that DQN agents trained to minimize queue congestion outperform greedy policies under high load. CSFD [7] embedded DRL inside the SDN controller for proactive congestion-aware routing. MAVTO [9] introduced GPS-based trajectory prediction to pre-position tasks at the vehicle's destination fog zone before handoff. Zhu et al. [8] combined DAG modeling, NSGA-II offline pre-training, and online meta-RL adaptation in a single pipeline.

Key limitation: No system in this generation combines multi-objective offline optimization, DRL online adaptation, SDN network management, DAG task modeling, and proactive GPS-driven mobility—all five capabilities simultaneously.

4 State of the Art

4.1 Multi-Objective Metaheuristic Optimization

NSGA-II remains the dominant metaheuristic for multi-objective task offloading. Jafari and Rezvani [1] established the canonical formulation: a population of routing chromosomes (arrays of fog/cloud decisions per task) is evolved using non-dominated sorting, crowding distance, SBX crossover, and MMDE mutation (9). Their evaluation on a 3-tier IoT–Fog–Cloud testbed demonstrated that Pareto front solutions provide meaningful energy–latency trade-off diversity, enabling operators to select operating points according to their QoS budget.

The key innovation of MMDE over standard polynomial mutation is directionality: the mutation vector $(r_2 - r_3)$ encodes population structure, steering mutations toward productive search regions rather than sampling uniformly at random. This produces faster convergence and more robust solutions under variable fog loads.

4.2 Task Pre-Filtering: The TOF-Broker

Nouri et al. [2] identified the boulder problem as the dominant failure mode of Generation-2 systems and proposed the TOF-Broker to address it. The broker applies EC classification (6) to each incoming task before any optimizer runs: boulders (EC $\geq 1.0\,\mathrm{s}$) are immediately routed to cloud; pebbles (EC $< 1.0\,\mathrm{s}$) enter the NSGA-II optimization pipeline. This achieves a 12.18% energy reduction with only 0.38% latency overhead, while eliminating fog node crashes under heavy-task workloads.

However, the TOF-Broker introduces the *Pebble Avalanche* problem: when task arrival rates exceed ∼1,000 tasks/second, the pebble queue grows faster than NSGA-II can drain it, eventually causing the same cascade failures the broker was designed to prevent. This remains an open problem addressed in Section 5.

4.3 Software-Defined Networking for Fog Systems

Gasmi and Ezzedine [3] conducted a systematic survey of SDN-based fog offloading, establishing the theoretical foundation for programmable network management in this domain. Their key finding: SDN's control/data plane separation—placing routing intelligence in a centralized controller while forwarding switches execute OpenFlow rules blindly—enables instant, globally consistent traffic reprogramming from a single API call, unavailable in traditional distributed routing.

This capability is critical for task offloading: when a large batch of pebbles needs to be sent to the cloud (a Super-Task), the SDN controller can open a dedicated high-priority lane on all intervening switches in under 5 ms, bypassing background telemetry traffic. However, Gasmi and Ezzedine [3] also identify the centralized controller bottleneck: under high IoT device density, the controller becomes overloaded with new-flow requests, adding 8–15 ms latency per flow—negating the benefit of SDN for latency-sensitive applications.

4.4 Microservice Architecture and DAG Task Modeling

Li et al. [4] surveyed 136 studies (2020–2024) and produced the first unified taxonomy bridging microservices orchestration and energy efficiency across five resource management pillars: service placement, resource provisioning, task scheduling and offloading, resource allocation, and replica selection. Their central finding is that inter-pillar synergy—jointly optimizing across all five pillars—remains almost entirely unaddressed in existing work.

The DAG model, formalized in [8] as $G = (\mathcal{V}, \mathcal{E})$ with per-node properties $\langle \ell_i, d_i^{\mathrm{in}}, d_i^{\mathrm{out}} \rangle$, enables partial offloading at step granularity: lightweight steps can execute on fog while GPU-heavy steps run on cloud, with data dependencies respected. The total application latency (8) now depends on inter-step data transfer costs, which vary depending on whether consecutive steps are co-located. This fundamentally changes the optimization problem—instead of one routing decision per task, there is one decision per DAG step.

4.5 Deep Reinforcement Learning for Task Offloading

4.5.1 DRL for Vehicular Fog Computing

An anonymous vehicular fog computing study [6] trained DQN agents on a grid-city simulation environment modeled on Salt Lake City's 200×200 m block structure. Client vehicles with insufficient compute offload tasks to nearby parked service vehicles acting

as fog nodes. The DQN observes queue depths, distances, and channel quality at all candidate nodes and selects the best destination via epsilon-greedy policy. Key result: DRL reduces queue congestion across the network—not just average per-task latency—and handles high-load scenarios where greedy methods cascade-fail.

However, this work has no energy objective, no offline pre-training (cold-start problem), and no mobility handling during task execution—all of which motivate the proposed architecture.

4.5.2 SDN-Integrated DRL: CSFD

The CSFD framework [7] embeds a DRL agent directly in the SDN control plane. Unlike standard SDN controllers that reactively install rules per new flow (8–15 ms overhead), CSFD's agent predicts upcoming traffic bursts and pre-installs OpenFlow rules before the traffic arrives—reducing per-flow overhead to zero. Evaluated in a VANET environment, CSFD achieves higher packet delivery ratio and lower latency than both DRL-only and SDN-only baselines. This work directly motivates the network-routing agent (Agent 2) in the proposed architecture; however, CSFD handles network routing only, with no compute placement intelligence—a critical gap.

4.5.3 DAG Task Offloading with Meta-RL and NSGA-II Pre-training

Zhu et al. [8] proposed the architecturally closest published system to the proposed PCNME. They model applications as DAGs, run NSGA-II offline on historical batches to generate Pareto-optimal routing plans, convert these plans into (state, action) training pairs, and pre-train a Meta-RL agent via behavioral cloning. The meta-policy adapts rapidly to new network environments via multiple MDPs, eliminating the cold-start problem. Their result: meta-RL combined with NSGA-II pre-training outperforms both stand-alone DRL and NSGA-II on dynamic environments.

Critically, this work has no mobility model and no SDN layer; their future work explicitly requests both—provided by the proposed architecture.

4.6 Mobility-Aware Offloading

4.6.1 Reactive Handoff: NTB/HTB Dual-Buffer System

A collaborative offloading framework [5] introduced the NTB/HTB dual-buffer system as a reactive fallback for mid-computation vehicle handoffs. When a vehicle disconnects from Fog A mid-task, the task migrates from the New Task Buffer (NTB) to the Handoff Task Buffer (HTB), which preempts the NTB and receives maximum CPU priority. Fog A completes the task, holds the result keyed by vehicle ID, and delivers it the moment the vehicle reconnects to any fog node—eliminating task re-submission. This achieves 50%+ latency reduction in high-mobility scenarios.

However, the system is purely reactive: it waits for physical disconnection before acting, wasting the entire handoff gap (typically 0.5–3 s) in wasted service time.

4.6.2 Proactive Handoff: MAVTO

MAVTO [9] introduced the first peer-reviewed proactive mobility framework for vehicular edge computing. Given a vehicle's Spatial Tag (GPS coordinates, speed v, heading θ), the

system computes:

$$T_{\text{exit}} = \frac{R_{\text{fog}} - \text{dist}(v_{\text{pos}},\, f_{\text{pos}})}{v_{\text{closing}}} \tag{10}$$

where v_{closing} is the component of vehicle velocity directed toward the zone boundary. MAVTO selects among three modes: *direct* ($T_{\text{exec}} < T_{\text{exit}}$—task finishes before vehicle leaves), *predictive* ($T_{\text{exec}} > T_{\text{exit}}$—pre-spin container at destination fog node), and *hybrid* (partial local execution). Evaluated on real vehicular mobility datasets, MAVTO achieves 25%+ improvement in task completion success rate over non-mobility-aware baselines.

Limitations: mode selection is rule-based (not learned), no multi-objective optimization, and no SDN routing for migration paths.

4.6.3 Spatiotemporal Prediction: Digital Twin + STGNN

The state of the art in trajectory prediction for vehicular edge computing is presented by Zhu et al. [10], who combine a Digital Twin (continuously synchronized virtual replica of the network), a Spatio-Temporal Graph Neural Network (STGNN) for demand forecasting, and A3C multi-agent DRL for task caching. The STGNN models the vehicular network as a graph $G = (\mathcal{N}, \mathcal{E})$ and applies graph convolution (spatial correlations) combined with LSTM (temporal patterns) to predict future task demand at each fog zone, enabling strategic resource pre-allocation 5–10 s ahead of actual arrivals.

This 2025 system represents the ceiling of current capabilities but lacks multi-objective optimization, task pre-filtering, and energy awareness.

4.7 Comparative Summary

Table 1 summarizes our ten primary references across six key dimensions. The proposed architecture (final row) is the only system addressing all six simultaneously.

Table 1: Comparative summary of surveyed works. ✓ = addressed, ○ = partial, × = not addressed.

Work	**Multi-obj.**	**Pre-filter**	**DAG**	**DRL**	**SDN**	**Mobility**	**Year**
Jafari & Rezvani [1]	✓	×	×	×	×	×	2021
Nouri et al. [2]	✓	✓	×	×	×	×	2024
Gasmi & Ezzedine [3]	×	×	×	×	✓	×	2023
Li et al. [4]	○	×	✓	○	×	×	2025
Collab. Framework [5]	×	×	×	×	×	○	2023
VFC-DRL [6]	×	×	×	✓	×	×	2024
CSFD [7]	×	×	×	✓	✓	×	2025
Zhu et al. DAG+MRL [8]	✓	×	✓	✓	×	×	2025
MAVTO [9]	×	×	×	×	×	✓	2024
Zhu et al. STGNN [10]	×	×	○	✓	×	✓	2025
Proposed PCNME	✓	✓	✓	✓	✓	✓	2026

5 Open Research Gaps

The survey reveals five clearly defined gaps. We formally state all five; the first two constitute our primary contributions to the gap analysis.

Gap 1 — The Integration Gap (Primary). No published system simultaneously addresses all six dimensions in Table 1: multi-objective optimization, pre-filtering, DAG task modeling, DRL online adaptation, SDN network management, and proactive mobility handling. Systems that advance one or two dimensions consistently ignore the others. The lack of integration is not incidental—it reflects the absence of a unified theoretical framework that models the interactions between compute placement, network routing, and mobility as a single joint optimization problem. Specifically, compute placement decisions (which fog node runs a DAG step) and network routing decisions (which physical path carries the step's data) are interdependent: a placement that minimizes compute latency may select a fog node whose network path is currently saturated, negating the benefit. Resolving this gap requires either joint optimization (intractable action space) or architectural coupling of two specialized agents with shared information—a design not yet demonstrated at the DAG-and-mobility scale.

Gap 2 — The Cold-Start Gap (Primary). All reviewed DRL systems are trained from random weight initialization, producing near-random policies during early deployment.

This cold-start period—spanning thousands to tens of thousands of interaction steps—causes significant service degradation that is unacceptable in production vehicular safety systems. Zhu et al. [8] demonstrate NSGA-II Pareto solutions as behavioral cloning data for meta-RL pre-training, but do not validate this pipeline in the full IoT–Fog–Cloud vehicular context with mobility and SDN. The critical open question is: can offline evolutionary optimization (TOF-MMDE-NSGA-II) systematically eliminate the cold-start problem for both compute placement and network routing agents, and does this pre-training advantage persist across the diverse environmental conditions of a city-scale vehicular network?

Gap 3 — The DAG-Mobility Gap. Mobility-aware offloading systems [5, 9] treat tasks as monolithic units. When a task is structured as a multi-step DAG, a handoff may occur between any two successive steps, requiring step-level migration decisions. No published work addresses proactive handoff for DAG-structured workloads where different steps may execute concurrently on different fog nodes.

Gap 4 — The Pebble Avalanche Gap. The TOF-Broker [2] resolves the boulder problem but introduces the Pebble Avalanche: under high task arrival rates, the pebble queue grows faster than NSGA-II drains it, causing cascade failures. No mechanism for graceful degradation under avalanche conditions (e.g., dynamic threshold adjustment or task aggregation and bulk cloud routing) has been proposed or validated.

Gap 5 — The Spatiotemporal Prediction Gap. MAVTO [9] uses linear dead-reckoning for trajectory prediction, which fails under sudden maneuvers, intersections, and variable speeds. Zhu et al. [10] demonstrate STGNN-based spatiotemporal demand prediction as a superior alternative, but do not integrate it with a proactive handoff mechanism or multi-objective optimization. A system combining STGNN prediction with proactive container migration and Pareto-optimal placement has not been demonstrated.

environment to

6 Proposed Architecture: Predictive Cloud-Native Mobile Edge

6.1 Architecture Overview

The Predictive Cloud-Native Mobile Edge (PCNME) architecture directly addresses Gaps 1–5 through five integrated components:

1. **TOF-Broker + Aggregator (Gaps 1, 4).** The TOF-Broker applies EC classification (6) per DAG step, routing boulders directly to cloud. The Aggregator handles avalanche conditions by compressing overflow pebbles into Super-Tasks routed via SDN VIP lanes.
2. **TOF-MMDE-NSGA-II Offline Optimizer (Gap 2).** Combines TOF pre-filtering with MMDE-enhanced NSGA-II (9) to generate Pareto-optimal routing plans from historical data. Plans are decomposed into (state, action) training pairs for behavioral cloning of both DRL agents, eliminating cold-start.
3. **DRL Agent 1 — Task Placement Brain (Gap 1).** A Deep Q-Network [6] pre-trained on NSGA-II Pareto solutions and fine-tuned online via experience replay.

Observes a 13-dimensional state vector (fog CPU loads, queue depths, task EC, bandwidth, vehicle speed, heading, T_{exit}, deadline remaining, cloud load). Action space: {FOG_A, FOG_B, FOG_C, FOG_D, CLOUD}.

4. **DRL Agent 2 — SDN Network Brain (Gaps 1, 4).** A second DQN embedded in the SDN controller [7], pre-trained on historical city traffic patterns. Pre-installs OpenFlow flow rules before traffic arrives, eliminating per-flow controller overhead. Decoupled from Agent 1 but shares network-state telemetry.

5. **Trajectory Predictor + NTB/HTB System (Gaps 3, 5).** Parses vehicle Spatial Tags (GPS, speed, heading) and computes T_{exit} (10) per DAG step individually—step-level mobility assessment, not task-level. Triggers proactive container pre-spin on the destination fog node when $T_{\text{exec}} > T_{\text{exit}}$. NTB/HTB buffers [5] provide deterministic fallback when trajectory prediction fails.

6.2 Hybrid Offline–Online AI Pipeline

Phase 1: Offline. Historical task logs and mobility traces feed TOF-MMDE-NSGA-II. Pareto solutions are converted to (state, action) pairs and used to pre-train both agents via behavioral cloning.

Phase 2: Online. Agent 1 makes placement decisions per pebble DAG step at inference speed (<1 ms). Agent 2 manages SDN flow rules proactively. Both agents update weights continuously from simulation rewards:

$$R = -0.5 \cdot L_{\text{norm}} - 0.3 \cdot E_{\text{norm}} - 0.2 \cdot \mathbb{1}[L > D] \cdot \lambda_{\text{penalty}}$$

where $\lambda_{\text{penalty}} = 10$ for safety-critical DAG steps.

7 Comparative Discussion

7.1 Against Generation-2 Systems

The proposed architecture subsumes [1] entirely: NSGA-II with MMDE is retained as the offline optimizer. The TOF-Broker (from [2]) reduces the problem NSGA-II must solve; the DRL agents provide real-time adaptation that NSGA-II cannot; the mobility layer handles vehicular dynamics neither predecessor addressed.

7.2 Against DAG+MRL [8]

Zhu et al.'s meta-RL system is the architecturally closest prior work. The proposed system extends it with: (i) TOF pre-filtering reducing Agent 1's decision space, (ii) SDN-managed network routing via Agent 2, and (iii) step-level proactive mobility via T_{exit} (10). Their future work explicitly requests all three.

7.3 Against CSFD [7]

CSFD provides state-of-the-art SDN+DRL network routing but has no compute placement agent. The proposed system's Agent 2 performs the same function as CSFD; Agent 1 provides the compute placement intelligence CSFD lacks. The architectural coupling between agents addresses Gap 1 in a way CSFD cannot.

7.4 Against MAVTO [9]

MAVTO provides the T_{exit} proactive handoff mechanism but applies it to monolithic tasks. The proposed system extends MAVTO with: (i) step-level assessment (Gap 3), (ii) RL-learned mode selection replacing MAVTO's rule-based thresholds, and (iii) SDN-managed migration paths for container transfer.

8 Conclusion

This literature review traced the evolution of task offloading research across four generations—from single-objective heuristics, through multi-objective evolutionary algorithms and hybrid pre-filtering systems, to the current generation of adaptive AI-driven architectures. We surveyed ten representative works spanning 2021–2025 and formally identified five open research gaps.

Gaps 1 and 2—the Integration Gap and Cold-Start Gap—represent our primary contributions to the gap analysis. Gap 1 establishes that the joint optimization of compute placement, network routing, and mobility under DAG task models has not been demonstrated at city scale. Gap 2 argues that the use of offline evolutionary Pareto solutions as pre-training data for online DRL agents is a promising and largely unvalidated approach to eliminating cold-start degradation in vehicular safety systems.

The proposed PCNME architecture addresses all five gaps through a unified pipeline: TOF-MMDE-NSGA-II offline optimization → dual DQN agent pre-training → live task placement and SDN routing with proactive GPS-driven mobility handling. No individual component is novel in isolation; the novelty lies in their integration and the offline-to-online knowledge transfer mechanism.

Future work directions include: (1) validating the behavioral cloning pre-training pipeline across diverse city topologies; (2) extending trajectory prediction from linear dead-reckoning to STGNN-based spatiotemporal forecasting [10]; (3) investigating federated learning to train Agent 2 across multiple city deployments without sharing raw traffic data; and (4) exploring TD3 or SAC as higher-performing alternatives to DQN for the continuous-action resource allocation sub-problem.

References

[1] Jafari, V. and Rezvani, M. H. Joint optimization of energy consumption and time delay in IoT–fog–cloud computing environments using NSGA-II metaheuristic algorithm. *Journal of Ambient Intelligence and Humanized Computing*, 14:1675–1698, 2023 (published online 2021).
URL: `https://link.springer.com/article/10.1007/s12652-021-03388-2`

[2] Nouri, N. and Rezvani, M. H. A multi-objective approach for optimizing IoT applications offloading in fog–cloud environments with NSGA-II. *The Journal of Supercomputing*, 2024.
URL: `https://link.springer.com/article/10.1007/s11227-024-06431-z`

[3] Gasmi, K. and Ezzedine, T. Reinforcement learning based task offloading of IoT applications in fog computing: algorithms and optimization techniques. *Cluster Computing*, 2024.

URL: https://www.ijcnis.org/index.php/ijcnis/article/view/8458/2522

[4] Li, Y. et al. Energy-efficient resource management in microservices-based fog and edge computing: state-of-the-art and future directions. *ACM Computing Surveys*, 2025. URL: https://arxiv.org/pdf/2512.04093

[5] Anonymous. A collaborative offloading task framework for IoT fog computing. Technical report, 2023.

[6] Anonymous. Deep reinforcement learning for delay-optimized task offloading in vehicular fog computing. *arXiv preprint arXiv:2410.03472*, 2024. URL: https://arxiv.org/html/2410.03472

[7] Anonymous. CSFD: Enhancing congestion reduction in vehicular communication with SDN and fog integration using deep reinforcement learning. *Analog Integrated Circuits and Signal Processing*, 2025.
URL: https://link.springer.com/article/10.1007/s10470-025-02541-7

[8] Zhu, L. et al. Dynamic task offloading scheme for edge computing via meta-reinforcement learning. *Journal of Manufacturing Systems*, 2025. URL: https://www.sciencedirect.com/science/article/pii/S1546221825000712

[9] Anonymous. Mobility-aware task offloading and resource allocation in UAV-assisted vehicular edge computing networks. *Drones*, 8(11):696, 2024.
URL: https://www.mdpi.com/2504-446X/8/11/696

[10] Zhu, L. et al. A vehicular edge computing offloading and task caching solution based on spatiotemporal prediction. *Future Generation Computer Systems*, 166:107679, 2025.
URL: https://www.sciencedirect.com/article/abs/pii/S0167739X24006435

[11] Mashal, I. et al. Multiobjective offloading optimization in fog computing using deep reinforcement learning. *Journal of Computer Networks and Communications*, 2024.